

Implementazione della classe `MathSet`



Obiettivo

L'obiettivo di questo esercizio è realizzare una classe in Kotlin chiamata `MathSet`, che rappresenta un **insieme matematico di numeri interi** e che supporti le principali operazioni insiemistiche.

Ricorda che un insieme, in matematica, è una collezione di elementi **non ordinata** in cui **non sono presenti elementi duplicati**. Alcuni esempi di insiemi validi:

- $A = \{1, 2, 3, 4, 5\}$
- $B = \{4, 1, -4, 10\}$
- $\mathbb{N}, \mathbb{Z}, \mathbb{Q}, \mathbb{R} \dots$ (insiemi numerici noti)



Requisiti Funzionali

La classe `MathSet` deve:

- Accettare una lista iniziale di numeri interi nel costruttore (`initialItems`).
- Evitare automaticamente i duplicati (gli insiemi non possono contenere elementi ripetuti).
- Fornire i seguenti metodi pubblici:



Funzionalità da implementare

Metodo	Descrizione
<code>isEmpty(): Boolean</code>	Verifica se l'insieme è vuoto. Restituisce <code>true</code> in caso affermativo, <code>false</code> altrimenti.
<code>equalsTo(other: MathSet): Boolean</code>	Determina l'uguaglianza tra due insiemi. Restituisce <code>true</code> in caso l'insieme <code>this</code> e l'insieme <code>other</code> sono uguali, <code>false</code> altrimenti. Ricorda! Due insiemi si dicono uguali quando hanno lo stesso numero di elementi e contengono gli stessi elementi Per esempio: $A = \{1, 2, 3\}$, $B = \{3, 1, 2\}$

Metodo	Descrizione
	$\implies A = B$
<code>contains(number: Int): Boolean</code>	Verifica se un numero è presente nell'insieme. Restituisce <code>true</code> in caso affermativo, <code>false</code> altrimenti.
<code>containsAll(numbers: List<Int>): Boolean</code>	Verifica se una lista di numeri è presente nell'insieme. Restituisce <code>true</code> se tutti gli elementi sono presenti, <code>false</code> se almeno uno non lo è.
<code>addItem(number: Int): Boolean</code>	Aggiunge un numero all'insieme se non è già presente. Restituisce <code>true</code> se l'aggiunta è avvenuta, <code>false</code> altrimenti.
<code>addItem(numbers: List<Int>): Boolean</code>	Aggiunge un elenco di numeri all'insieme — evitando di inserire elementi duplicati. Restituisce <code>true</code> se tutti i numeri sono stati correttamente inseriti, <code>false</code> se almeno un numero non è stato inserito.
<code>removeItem(number: Int): Boolean</code>	Rimuove un numero dall'insieme, se presente. Restituisce <code>true</code> se rimosso, <code>false</code> se non esisteva.
<code>getAll(): List<Int></code>	Restituisce una nuova lista con tutti gli elementi presenti nell'insieme.
<code>union(other: MathSet): MathSet</code>	Restituisce un nuovo <code>MathSet</code> che rappresenta l'unione tra questo insieme e un altro. · Richiamo alla teoria: Presi due insiemi $A = \{1, 2, 3\}$ e $B = \{1, 6, 7\}$, l'unione tra i due restituisce: $A \cup B = \{1, 2, 3, 6, 7\}$
<code>intersection(other: MathSet): MathSet</code>	Restituisce un nuovo <code>MathSet</code> che contiene solo gli elementi in comune tra i due insiemi. · Richiamo alla teoria: Presi due insiemi $A = \{1, 2, 3\}$ e $B = \{1, 6, 7\}$, l'intersezione tra i due restituisce: $A \cap B = \{1\}$



Proprietà

- `itemsCount` : proprietà `val` che restituisce il numero di elementi presenti nell'insieme.



Test

Scrivi una suite di test automatici (con **JUnit 5**) che verifichi il comportamento di **tutti** i metodi pubblici.

Esempi di casi da testare:

- Aggiunta di elementi duplicati.
- Rimozione di elementi non presenti.
- Unione tra insiemi con e senza elementi comuni.
- Intersezione tra insiemi con e senza elementi comuni.
- Verifica del conteggio totale degli elementi (`itemsCount`).

Suggerimenti

- Usa una `MutableList` internamente, ma assicurati di controllare i duplicati manualmente.
- Non usare strutture come `Set` , l'obiettivo è esercitarsi con la logica di base.
- Presta attenzione alla gestione delle copie negli oggetti restituiti (ad esempio in `getAll()`).